

Qt Core: A 30-Day Practical Curriculum

Beginner to Advanced — Systems & IoT Focus

Compiled for George

July 2026

Qt Core — 30-Day Curriculum

Day 1: Qt's Object Model — QObject, the Meta-Object System, and moc

Day 2: QApplication & the Event Loop

Day 3: Signals & Slots — Mechanism and Connection Types

Day 4: QTimer — Single-shot, Repeating, and Precision Modes

Day 5: QObject Parent/Child Ownership & Memory Management

Day 7: Mini-Project — Log-Rotating File Writer Driven by QTimer

Day 8: QFile, QIODevice, QTextStream, QDataStream

Day 9: QSettings — Configuration Files Done Properly

Day 10: QJsonDocument / QJsonObject / QJsonArray

Day 11: QRegularExpression — Parsing Serial Line Formats Properly

Day 12: Q_PROPERTY in Depth & Custom QObject-Derived Data Model
Classes

Day 13: QDateTime & QTimeZone — Timestamping Correctly

Day 14: Mini-Project — Serial-Line Parser → Structured JSON Record Pipeline

Day 16: The QThread + moveToThread Worker Pattern, Fully Formalized

Day 17: QMutex & QReadWriteLock — Shared Data Between Threads

Day 18: QThreadPool & QRunnable — Task-Based Concurrency

Day 19: QProcess — Spawning and Communicating with External Processes

Day 20: QFileSystemWatcher — Reacting Live to File/Config Changes

Day 21: Mini-Project — Multi-Threaded Worker Pool Processing Simulated Sensor Readings

Day 23: QUdpSocket — Connectionless, Datagram-Based Communication

Day 24: QNetworkAccessManager — HTTP Requests

Day 25: QStateMachine — Formal State Machines for Connection Lifecycles

Day 26: Event Filters & Custom QEvent Subclasses

Day 27: QLoggingCategory & Idiomatic Qt Error Handling

Day 28: QTest — Unit Testing Qt Core Code, Including Async/Signal-Driven Code

Day 29: CMake Project Structure & Deployment for Embedded Linux Targets

Day 30 (Capstone): Qt Core Serial → JSON → Network Relay

Qt Core — 30-Day Curriculum

Format: Concept explanation → annotated, fully resolved code → key takeaways. Framed throughout around an MQTT device-monitoring context (`mqtt_monitor`), consistent with prior C++, Python, Docker, and PHP curricula.

Syllabus

Week 1 — Foundations 1. Qt's object model: QObject, meta-object system, moc 2. QApplication & the event loop 3. Signals & slots — mechanism and connection types 4. QTimer — single-shot, repeating, precise timers 5. QObject parent/child ownership & memory management 6. Qt containers (QString, QList, QMap, QVariant) vs STL 7. Mini-project: log-rotating file writer driven by QTimer

Week 2 — Data & Configuration 8. QFile, QIODevice, QTextStream, QDataStream 9. QSettings (ini/native config) 10. QJsonDocument / QJsonObject / QJsonArray 11. QRegularExpression 12. Q_PROPERTY and custom data model classes 13. QDateTime, QTimeZone 14. Mini-project: serial-line parser → structured JSON record

Week 3 — Concurrency & Processes 15. Qt's threading model: thread affinity, per-thread event loops 16. QThread + moveToThread worker pattern 17. QMutex, QReadWriteLock 18. QThreadPool & QRunnable 19. QProcess 20. QFileSystemWatcher 21. Mini-project: multi-threaded worker pool processing simulated sensor readings

Week 4 — Networking, State, Integration 22. QTcpSocket / QTcpServer 23. QUdpSocket 24. QNetworkAccessManager 25. QStateMachine 26. Event filters & custom QEvent subclasses 27. QLoggingCategory & idiomatic Qt error handling 28. QTest unit testing 29. CMake structure & deployment for embedded targets 30. Capstone: Qt Core serial→JSON→network relay

Day 1: Qt's Object Model — QObject, the Meta-Object System, and moc

Theory: why Qt needed its own object model

Standard C++ has no built-in reflection — at runtime an object doesn't know its own class name, its list of methods, or which of its members are meant to notify others of change. Qt needed all of that to make signals/slots, properties, and dynamic introspection work, without waiting for C++ to grow real reflection. Qt's solution, still in use today:

1. **QObject** — the base class every “Qt-aware” class derives from. It gives an object an identity in Qt's world: a parent/child tree, an event-handling interface, and a `QMetaObject*` describing its own type.
2. **The `Q_OBJECT` macro** — declares (but does not define) extra methods the meta-object system needs: `metaObject()`, `qt_metacast()`, `qt_metacall()`, plus storage for the class's static `QMetaObject` instance.
3. **moc (Meta-Object Compiler)** — a code generator that runs before your normal C++ compiler. It scans headers for `Q_OBJECT`, `signals:`, `slots:`, and `Q_PROPERTY`, and generates a `moc_yourclass.cpp` file containing the real implementations of those methods, including signal emission machinery and reflection tables.

Signals and slots are not a C++ language feature — they're ordinary C++ methods with generated plumbing. `emit mySignal(x)` expands into a call to moc-generated code that looks up connections in an internal table and invokes connected slots.

Why CMake's `AUTOMOC ON` matters: it runs moc automatically on every header with `Q_OBJECT` before compiling. Forgetting `Q_OBJECT`, or forgetting `AUTOMOC`, produces a compiler error about a missing meta-object member (if `Q_PROPERTY` is used without `Q_OBJECT`), or a linker error like `undefined reference to vtable for YourClass` (if moc never ran). Recognizing these two specific errors immediately — rather than debugging class logic — saves real time.

Resolved example — reflection proven directly:

```

// device.h
#pragma once
#include <QObject>
#include <QString>

class Device : public QObject
{
    Q_OBJECT    // triggers moc to generate metaObject(), qt_metacast(), etc.

    Q_PROPERTY(QString name READ name WRITE setName)

public:
    explicit Device(const QString &name, QObject *parent = nullptr)
        : QObject(parent), m_name(name) {}

    QString name() const { return m_name; }
    void setName(const QString &name) { m_name = name; }

private:
    QString m_name;
};

```

```

// main.cpp
#include <QCoreApplication>
#include <QDebug>
#include "device.h"

int main(int argc, char *argv[])
{
    QCoreApplication app(argc, argv);
    Device sensor("TempSensor1");

    const QMetaObject *meta = sensor.metaObject();
    qDebug() << "Class name via reflection:" << meta->className();
    // Output: Device (from moc-generated code, not typeid(sensor).name())

    qDebug() << "Property count:" << meta->propertyCount();
    for (int i = 0; i < meta->propertyCount(); ++i)
        qDebug() << " - " << meta->property(i).name();
    // Output includes: objectName, name

    sensor.setProperty("name", "TempSensor1-Renamed"); // set purely by string name
    qDebug() << "After setProperty by string:" << sensor.name();
    // Output: TempSensor1-Renamed

    return 0;
}

```

CMakeLists.txt:

```
cmake_minimum_required(VERSION 3.16)
project(day1)
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_AUTOMOC ON)
find_package(Qt6 REQUIRED COMPONENTS Core)
add_executable(day1 main.cpp device.h)
target_link_libraries(day1 Qt6::Core)
```

Deliberate failure, resolved: removing `Q_OBJECT` but keeping `Q_PROPERTY` gives a compiler error ('staticMetaObject' is not a member of 'Device'). Keeping `Q_OBJECT` but never running moc gives a linker error (undefined reference to vtable for Device). **Compiler error mentioning a missing meta-object member → check `Q_OBJECT`. Linker error mentioning a missing vtable → check moc actually ran.**

Key takeaways - QObject + `Q_OBJECT` + moc give C++ a lightweight, hand-rolled reflection system. - Signals/slots and `Q_PROPERTY` are macros and generated code on top of ordinary C++, not special syntax. - The meta-object system is genuinely introspectable at runtime. - Missing `Q_OBJECT` → compiler error. Missing moc generation → linker error about a vtable.

Day 2: QApplication & the Event Loop

Theory: what an “event” is, and why the loop exists

Qt Core programs spend almost all their life inside `QCoreApplication::exec()`, which repeatedly: (1) checks the platform’s event queue, (2) wraps each occurrence in a `QEvent`, (3) calls `notify(receiver, event) → receiver->event(event)`, a virtual method every `QObject` has, and (4) returns to step 1, sleeping efficiently when idle. This is single-threaded cooperative multitasking: blocking a slot means nothing else — no timer, no socket — gets serviced until you return control, because step 3 doesn’t return until your handler does.

Resolved example 1 — proving the loop is the actual runtime:

```
#include <QCoreApplication>
#include <QTimer>
#include <QDebug>

int main(int argc, char *argv[])
{
    QCoreApplication app(argc, argv);

    qDebug() << "Before exec() -- runs immediately, outside the loop";

    QTimer::singleShot(2000, [] {
        qDebug() << "Inside the loop -- fired after 2000ms";
        qApp->quit();
    });

    qDebug() << "Calling exec() now -- program blocks here";
    int result = app.exec();

    qDebug() << "exec() returned, result =" << result;
    return result;
}
```

Output, in exact order:

```
Before exec() -- runs immediately, outside the loop
Calling exec() now -- program blocks here
Inside the loop -- fired after 2000ms
exec() returned, result = 0
```

Resolved example 2 — blocking the loop and its cost:

```

#include <QCoreApplication>
#include <QTimer>
#include <QDebug>
#include <QThread>

int main(int argc, char *argv[])
{
    QCoreApplication app(argc, argv);
    QTimer heartbeat;
    int tick = 0;

    QObject::connect(&heartbeat, &QTimer::timeout, [&tick] {
        ++tick;
        qDebug() << "heartbeat tick" << tick;
        if (tick == 3) {
            qDebug() << "Simulating a blocking operation for 4 seconds...";
            QThread::sleep(4); // BAD: blocks the thread the event loop runs on
            qDebug() << "...blocking operation finished";
        }
    });
    heartbeat.start(1000);

    QTimer::singleShot(8000, [] { qApp->quit(); });
    return app.exec();
}

```

Ticks 1–3 fire on schedule; then a 4-second gap during which **no timer fires at all**, because the event loop itself is suspended — `QThread::sleep()` blocks the thread, not just “that slot.” Tick 4 fires immediately once the sleep ends, then normal spacing resumes. This is the mechanism behind “my Qt service occasionally stalls” bugs: some slot is doing blocking work on the main thread.

Resolved example 3 — graceful shutdown on SIGINT/SIGTERM:


```

#include <QCoreApplication>
#include <QTimer>
#include <QDebug>
#include <csignal>

void handleTermination(int signum)
{
    qDebug() << "Received signal" << signum << "-- requesting clean shutdown";
    if (qApp) qApp->quit(); // safe: quit() just posts a QEvent::Quit
}

int main(int argc, char *argv[])
{
    QCoreApplication app(argc, argv);
    std::signal(SIGINT, handleTermination);
    std::signal(SIGTERM, handleTermination);

    QTimer heartbeat;
    QObject::connect(&heartbeat, &QTimer::timeout, [] { qDebug() << "service alive"; });
    heartbeat.start(1000);

    int result = app.exec();
    qDebug() << "Clean shutdown complete, exit code" << result;
    return result;
}

```

`kill -TERM <pid>` stops the service cleanly (code 0); `kill -9` gives it no chance to reach cleanup at all — relevant later for systemd (Day 29).

Key takeaways - `exec()` is the program’s execution model until `quit()`, driven by dispatched events. - Every timer, queued signal, and I/O readiness notification funnels through the same dispatch mechanism. - Blocking any code on the event-loop thread freezes every timer and event on that thread — the root cause of most “my Qt service hangs intermittently” bugs. - Signal handlers should do the minimum — calling `quit()` is the documented-safe pattern for graceful shutdown under systemd.

Day 3: Signals & Slots — Mechanism and Connection Types

Theory: what actually happens on connect() and emit

`QObject::connect(sender, signal, receiver, slot)` registers an entry in an internal table. `emit signal(args)` walks that table and invokes each connected slot according to its **connection type**:

- **Qt::DirectConnection** — slot called immediately, synchronously, on whatever thread `emit` ran on.
- **Qt::QueuedConnection** — the call is packaged as a `QMetaCallEvent` and posted to the receiver's thread's event queue; executes later, when that thread's event loop processes it.
- **Qt::AutoConnection** (default) — same thread → Direct; different threads → Queued, auto-detected at connect time.

Resolved example 1 — one signal, multiple slots; one slot, multiple signals:

```
// sensor.h
#pragma once
#include <QObject>

class Sensor : public QObject
{
    Q_OBJECT
public:
    explicit Sensor(QObject *parent = nullptr) : QObject(parent) {}

    void takeReading(double value)
    {
        m_lastValue = value;
        emit readingChanged(value);
        if (value > m_threshold) emit thresholdExceeded(value);
    }
    void setThreshold(double t) { m_threshold = t; }

signals:
    void readingChanged(double value);
    void thresholdExceeded(double value);

private:
    double m_lastValue = 0.0;
    double m_threshold = 30.0;
};
```

```

#include <QCoreApplication>
#include <QDebug>
#include "sensor.h"

void logReading(double v) { qDebug() << "[LOG] reading:" << v; }

int main(int argc, char *argv[])
{
    QCoreApplication app(argc, argv);
    Sensor sensor;
    sensor.setThreshold(30.0);

    QObject::connect(&sensor, &Sensor::readingChanged, logReading);
    QObject::connect(&sensor, &Sensor::readingChanged, [](double v) {
        qDebug() << "[UI] would update display to:" << v;
    });
    auto alert = [](double v) { qDebug() << "[ALERT] threshold event, value =" << v; };
    QObject::connect(&sensor, &Sensor::thresholdExceeded, alert);

    sensor.takeReading(22.0);
    sensor.takeReading(35.0);
    return 0;
}

```

Output:

```

[LOG] reading: 22
[UI] would update display to: 22
[LOG] reading: 35
[UI] would update display to: 35
[ALERT] threshold event, value = 35

```

Resolved example 2 — connection type made visible:

```

QObject::connect(&emitter, &Emitter::ping, &emitter, [] {
    qDebug() << "direct slot running";
}, Qt::DirectConnection);

QObject::connect(&emitter, &Emitter::ping, &emitter, [] {
    qDebug() << "queued slot running";
}, Qt::QueuedConnection);

qDebug() << "before fire()";
emitter.fire();
qDebug() << "after fire(), before exec()";

```

Output:

```

before fire()
direct slot running
after fire(), before exec()
queued slot running

```

`fire()` returns before the queued slot ever runs — a queued connection posts an event and moves on; the direct slot executes synchronously, finishing before `fire()` returns.

Resolved example 3 — disconnecting and automatic connection teardown:

```
QMetaObject::Connection conn = QObject::connect(&sensor, &Sensor::readingChanged, [](double
v) {
    qDebug() << "listener A saw:" << v;
});
QObject::connect(&sensor, &Sensor::readingChanged, [](double v) {
    qDebug() << "listener B saw:" << v;
});

sensor.takeReading(10.0);    // both fire
QObject::disconnect(conn);  // remove listener A only
sensor.takeReading(20.0);    // only B fires
```

If `sensor` were destroyed instead, every connection where it's sender or receiver is automatically removed — no manual disconnect bookkeeping needed for object lifetime, only for surgical removal while both objects remain alive.

Key takeaways - `connect()` registers a table entry; `emit` walks it and invokes slots per connection type. - Direct = synchronous inline call; Queued = deferred via the event loop; Auto = picks based on thread affinity. - One signal → many slots; one slot ← many signals — zero compile-time coupling. - `connect()` returns a handle for `disconnect()`; `QObject` destruction auto-severs all its connections.

Day 4: QTimer — Single-shot, Repeating, and Precision Modes

Theory: QTimer is a QObject using everything already learned

`QTimer` is a `QObject`; when started it registers with the platform timer facility, and when the interval elapses, the event loop delivers a timer event that `QTimer::event()` translates into the `timeout()` signal.

Timer types: - `Qt::PreciseTimer` — ~1ms accuracy, more expensive. Use for protocol timeouts, precise sampling. - `Qt::CoarseTimer` (default) — ~5% accuracy, cheaper; OS can batch wake-ups (good for battery-powered embedded targets). Right choice for heartbeats, periodic housekeeping. - `Qt::VeryCoarseTimer` — ~1s accuracy, fine for “flush logs once a minute.”

Resolved example 1 — repeating timer, single-shot, stop mid-run:

```
QTimer pollTimer;
pollTimer.setInterval(500);
pollTimer.setTimerType(Qt::PreciseTimer);
pollTimer.setSingleShot(false); // repeating (also the default)

int tickCount = 0;
QObject::connect(&pollTimer, &QTimer::timeout, [&]() {
    qDebug() << "poll tick" << ++tickCount;
    if (tickCount == 5) pollTimer.stop(); // halts future emissions; object still valid
});
pollTimer.start();

QTimer::singleShot(3000, [] { qDebug() << "one-off event at 3s mark"; });
```

`stop()` doesn't destroy the `QTimer` — it can be `start()`ed again, useful for pausing/resuming polling without rebuilding the object.

Resolved example 2 — coarse vs precise, observably different:

Precise ticks land tightly on 100ms multiples; coarse ticks visibly drift/batch, because the OS is intentionally allowed to slide coarse timers to align with other pending wake-ups. For fixed-protocol serial polling, use `PreciseTimer`; for “flush a buffer every few seconds,” `CoarseTimer` costs less and nothing depends on exact timing.

Resolved example 3 — drift-corrected periodic task:

A naive repeating timer accumulates drift because each interval starts counting only after the previous slot finishes. The fix: re-anchor against the original start time.

```
class DriftCorrectedPoller : public QObject
{
    Q_OBJECT
public:
    explicit DriftCorrectedPoller(int intervalMs, QObject *parent = nullptr)
        : QObject(parent), m_intervalMs(intervalMs)
    {
        connect(&m_timer, &QTimer::timeout, this, &DriftCorrectedPoller::onTick);
    }
    void start()
    {
        m_startTime = QDateTime::currentMSecsSinceEpoch();
        m_tickNumber = 0;
        scheduleNext();
    }
private slots:
    void onTick()
    {
        ++m_tickNumber;
        qint64 now = QDateTime::currentMSecsSinceEpoch();
        qDebug() << "tick" << m_tickNumber << "offset:" << (now - m_startTime)
                << "ideal:" << m_tickNumber * m_intervalMs;
        scheduleNext();
    }
    void scheduleNext()
    {
        qint64 idealNextTime = m_startTime + (m_tickNumber + 1) * m_intervalMs;
        qint64 delay = idealNextTime - QDateTime::currentMSecsSinceEpoch();
        m_timer.setSingleShot(true);
        m_timer.start(qMax<qint64>(delay, 0));
    }
private:
    QTimer m_timer;
    int m_intervalMs;
    qint64 m_startTime = 0;
    int m_tickNumber = 0;
};
```

Each `scheduleNext()` re-anchors against `m_startTime`, not “now” — error stays within one tick indefinitely instead of compounding over an hour of runtime. Note: a `QObject` class defined directly in a `.cpp` needs `#include "main.moc"` at the bottom.

Key takeaways - `timeout()` is delivered through the same event-loop/signal mechanism as everything else — nothing new under the hood. - `setSingleShot(false)` repeats; `true` fires once. `stop()` halts emissions without destroying the object. - Precise vs coarse is a real, measurable accuracy/efficiency

trade-off — pick deliberately. - Naive repeating timers drift because each interval is scheduled relative to “now” — re-anchor to an absolute schedule for long-running periodic tasks.

Day 5: QObject Parent/Child Ownership & Memory Management

Theory: Qt's tree-based ownership vs RAII/smart pointers

Every `QObject` constructor takes an optional parent. If passed, the child registers in the parent's children list. When the parent is destroyed, its destructor deletes every child recursively. A child removes itself from its parent's list if destroyed independently first. **If an object has a parent, you almost never call `delete` on it yourself.**

Critical rule: never give a heap object two parents, and never mix raw heap ownership with `unique_ptr` for anything with a `QObject` parent.

Resolved example 1 — cascading deletion, proven with destructor logging:

```
class Component : public QObject
{
    Q_OBJECT
public:
    Component(const QString &name, QObject *parent = nullptr) : QObject(parent), m_name(name)
    { qDebug() << "constructing" << m_name; }
    ~Component() override { qDebug() << "destroying" << m_name; }
private:
    QString m_name;
};

// usage:
auto *reader = new Component("SerialReader", nullptr);
auto *parser = new Component("Parser", reader);
auto *logger = new Component("Logger", parser);

delete reader;    // cascades: reader destroys parser, parser destroys logger
```

Output:

```
constructing SerialReader
constructing Parser
constructing Logger
destroying SerialReader
destroying Parser
destroying Logger
```

One `delete` call destroyed three objects, root-to-leaf.

Resolved example 2 — the real double-delete danger, and the fix:


```

// REAL broken pattern:
auto *parent = new Component("Parent", nullptr);
auto *child = new Component("Child", parent);
delete parent;    // deletes 'child' as part of cascading destruction
delete child;     // CRASH: child's memory already freed

// FIX: never hold a pointer to a child past its parent's destruction
auto *parent2 = new Component("Parent", nullptr);
new Component("Child", parent2);    // don't keep the pointer if not needed independently
delete parent2;    // single delete, cascades correctly

```

The bug isn't "deleting a child" — it's holding a pointer to something whose owner might delete it first.

Resolved example 3 — QPointer: a safe, auto-nulling weak reference:

```

auto *parent = new Component("Parent");
auto *rawChildPtr = new Component("Child", parent);
QPointer<Component> safeChildRef(rawChildPtr);    // does NOT take ownership

QDebug() << safeChildRef.isNull();    // false
delete parent;                        // destroys parent AND child
QDebug() << safeChildRef.isNull();    // true -- safe to check

if (safeChildRef) { /* never reached */ }
else { qDebug() << "safely detected the child is gone"; }

```

`QPointer<T>` doesn't extend lifetime (unlike `shared_ptr`) but is automatically zeroed when the observed `QObject` is destroyed — the correct tool for holding-but-not-owning.

Key takeaways - Passing a parent hands ownership to it; its destructor deletes children recursively. - The real danger is holding a raw pointer to a child past its parent's lifetime — fix via not holding such pointers, or `QPointer<T>`. - Don't mix Qt's tree ownership with `unique_ptr`/`shared_ptr` for the same object. - Parentless objects are your responsibility to delete — typically graph roots or C++-RAII-managed objects outside Qt's tree. ## Day 6: Qt Containers — `QString`, `QList`, `QMap`, `QVariant`, and Implicit Sharing

Theory: implicit sharing (copy-on-write)

Qt containers (`QString`, `QList`, `QMap`, `QByteArray`) use implicit sharing: copying is a cheap reference-count bump; the deep copy only happens lazily, the moment one copy is modified ("detach"). Passing by value is not the performance concern it is with STL. This has real implications for multi-threaded code — reference counting is thread-safe, but concurrent reads/writes on what look like separate copies can race until a write triggers a detach.

Resolved example 1 — proving COW with QList:

```
QList<int> original = {1, 2, 3, 4, 5};
QList<int> copy = original;    // NOT a deep copy -- shares the buffer

QDebug() << original.constData() == copy.constData();    // true: identical pointer

copy.append(6);    // WRITE triggers detach here

QDebug() << original.constData() == copy.constData();    // false: copy now has its own buffer
QDebug() << original;    // (1,2,3,4,5) -- unaffected
QDebug() << copy;    // (1,2,3,4,5,6)
```

Assignment is $O(1)$; the $O(n)$ deep copy only happens at the moment of mutation. Reading never triggers a detach; only non-const mutation does.

Resolved example 2 — QString, and the const-correctness trap:

```
void processReadOnly(const QString &s) { qDebug() << s.length(); }    // no detach, ever

QString processAndModify(QString s) {
    s[0] = 'X';    // non-const operator[] forces a detach
    return s;
}

QString original = "TEMP:23.5";
processReadOnly(original);
QString modified = processAndModify(original);
// original: "TEMP:23.5" (unaffected), modified: "Xemp:23.5"
```

Avoid gratuitous non-const method calls (`operator[]`, non-const `data()`) on containers you only intend to read — each risks an unnecessary detach, a real cost in a hot path like a serial-line parser called thousands of times per second.

Resolved example 3 — QMap for structured lookups, QVariant as type-erased glue:

```

 QMap<QString, QVariant> deviceReading;
 deviceReading["device_id"] = "sensor-07";
 deviceReading["temperature"] = 23.5;
 deviceReading["humidity"] = 61;
 deviceReading["online"] = true;

 // QMap iterates in SORTED key order (red-black tree) -- deterministic, unlike QHash

 bool ok = false;
 double temp = deviceReading.value("temperature").toDouble(&ok); // 23.5, ok=true

 int bad = deviceReading.value("device_id").toInt(&ok); // 0, ok=false -- ALWAYS check ok

 QVariant missing = deviceReading.value("pressure"); // key doesn't exist
 qDebug() << missing.isValid(); // false

```

`QMap` guarantees sorted iteration (useful for deterministic JSON key order or repeatable test output); `QHash` is unordered but faster on average.

Key takeaways - Qt containers use COW: copies are O(1); deep copies happen lazily on first write. - Prefer `const&` parameters and `const` accessors to avoid needless detaches. - `QMap` keeps keys sorted (deterministic); `QHash` doesn't, but is faster average-case. - Always check the `bool *ok` out-parameter on `QVariant::toXxx()` conversions — a failed conversion silently returns a default rather than throwing.

Day 7: Mini-Project — Log-Rotating File Writer Driven by QTimer

Synthesizes QObject ownership (Day 5), signals/slots (Day 3), QTimer (Day 4), and containers (Day 6) into a component that buffers log lines, flushes on a timer, and rotates files past a size threshold.

```

// rotatinglogwriter.h
class RotatingLogWriter : public QObject
{
    Q_OBJECT
public:
    explicit RotatingLogWriter(const QString &directory, qint64 maxBytesPerFile = 4096,
                               int flushIntervalMs = 2000, QObject *parent = nullptr)
        : QObject(parent), m_directory(directory), m_maxBytesPerFile(maxBytesPerFile)
    {
        QDir().mkpath(m_directory);
        m_flushTimer.setInterval(flushIntervalMs);
        m_flushTimer.setTimerType(Qt::CoarseTimer); // flushing isn't timing-critical
        connect(&m_flushTimer, &QTimer::timeout, this, &RotatingLogWriter::flush);
        m_flushTimer.start();
        openNewFile();
    }
    ~RotatingLogWriter() override { flush(); if (m_currentFile.isOpen())
m_currentFile.close(); }

    public slots:
        void writeLine(const QString &line)
        {
            m_buffer.append(QDateTime::currentDateTime().toString(Qt::ISODate) + " " + line);
        }

    signals:
        void fileRotated(const QString &newPath);

    private slots:
        void flush()
        {
            if (m_buffer.isEmpty()) return; // avoid a pointless disk write
            QTextStream out(&m_currentFile);
            for (const QString &line : std::as_const(m_buffer)) out << line << '\n';
            out.flush();
            m_buffer.clear();
            if (m_currentFile.size() >= m_maxBytesPerFile) rotate();
        }
        void rotate()
        {
            m_currentFile.close();
            openNewFile();
            emit fileRotated(m_currentFile.fileName());
        }

    private:
        void openNewFile()
        {
            const QString name = QString("log_%1.txt")
                .arg(QDateTime::currentDateTime().toString("yyyyMMdd_hhmmss_zzz"));
            m_currentFile.setFileName(m_directory + "/" + name);
            if (!m_currentFile.open(QIODevice::WriteOnly | QIODevice::Text))
                qWarning() << "Failed to open log file:" << m_currentFile.fileName();
        }
        QString m_directory;
        qint64 m_maxBytesPerFile;
        QStringList m_buffer;
        QFile m_currentFile;

```

```
        QTimer m_flushTimer;  
    };
```

Driver code generates fake lines every 150ms, faster than the 1000ms flush interval, proving buffering, batched flush, and rotation all work together. Manual `delete writer` at shutdown (it has no parent) guarantees its destructor's `flush()` runs before exit, so nothing buffered is lost.

Design rationale: - Buffering + timer-driven flush avoids doing a slow disk write inside a high-frequency slot (Day 2's blocking-loop warning). - `QStringList::clear()` after flush is cheap (Day 6) since nothing else references the buffer. - `Qt::CoarseTimer` is correct here since flush timing has no millisecond-level correctness requirement (Day 4). - Explicit destructor flush is required for any buffering component — otherwise data still in the buffer is lost on shutdown.

Day 8: QFile, QIODevice, QTextStream, QDataStream

Theory: QIODevice — the abstraction unifying files, sockets, processes

`QFile` is one of several `QIODevice` subclasses. `QTcpSocket`, `QProcess`, and `QFile` all share the same interface: `open()`, `close()`, `read()`, `write()`, `readyRead()`. Code written against `QIODevice/QTextStream` generalizes directly to sockets and processes later.

- **QTextStream** — text: encoding, line-based reading, formatted output.
- **QDataStream** — binary: versioned, typed serialization for compact structured storage.

Resolved example 1 — QFile + QTextStream, write and read:

```
QFile file("readings.txt");
file.open(QIODevice::WriteOnly | QIODevice::Text);
QTextStream out(&file);
out << "TEMP:23.5\n" << "TEMP:24.1\n" << "HUMIDITY:61\n";
// (file closes automatically when it goes out of scope)

QFile in_file("readings.txt");
in_file.open(QIODevice::ReadOnly | QIODevice::Text);
QTextStream in(&in_file);
while (!in.atEnd()) qDebug() << in.readLine();
```

Resolved example 2 — error handling, always checked:

```
QFile blocked("/root/should_fail.txt");
if (!blocked.open(QIODevice::WriteOnly)) {
    qDebug() << "error code:" << blocked.error();
    qDebug() << "error string:" << blocked.errorString(); // "Permission denied"
}
```

Every `open()` returns `bool` and must be checked — there's no excuse for ignoring a failed open in unattended service code.

Resolved example 3 — QDataStream, versioned binary round-trip:

```

struct Reading { QString deviceId; double value; qint64 timestampMs; };

QDataStream &operator<<(QDataStream &out, const Reading &r)
{ out << r.deviceId << r.value << r.timestampMs; return out; }
QDataStream &operator>>(QDataStream &in, Reading &r)
{ in >> r.deviceId >> r.value >> r.timestampMs; return in; }

// write:
QFile file("readings.bin");
file.open(QIODevice::WriteOnly);
QDataStream out(&file);
out.setVersion(QDataStream::Qt_6_5); // ALWAYS pin a version explicitly
out << readings.size();
for (const Reading &r : readings) out << r;

// read: (same setVersion, symmetric read)

```

Pinning `setVersion()` freezes the binary format regardless of the reading/writing Qt point release used later.

Key takeaways - `QFile`, `QTcpSocket`, `QProcess` all derive from `QIODevice` — code generalizes across all three. - `QTextStream` for human-readable/interoperable text; `QDataStream` for compact typed binary. - Every `open()` call returns `bool` and must be checked with `errorString()` on failure. - Custom types need explicit `operator<</operator>>` for `QDataStream`; always pin `setVersion()`.

Day 9: QSettings — Configuration Files Done Properly

Theory: `QSettings` gives typed round-tripping via `QVariant`, hierarchical grouping (`beginGroup()` / `endGroup()`), and platform-appropriate or explicit-path storage. Use `QSettings::IniFormat` with an explicit path for deployed services.

Resolved example — typed config, with sensible defaults:

```
QSettings settings("mqtt_monitor.ini", QSettings::IniFormat);
settings.beginGroup("mqtt");
settings.setValue("broker_host", "192.168.1.50");
settings.setValue("broker_port", 1883);
settings.endGroup();
settings.sync();

// reading:
settings.beginGroup("mqtt");
QString host = settings.value("broker_host", "localhost").toString();
int port = settings.value("broker_port", 1883).toInt();
settings.endGroup();

int missingRetries = settings.value("network/max_retries",
3).toInt(); // default used, key never set
```

`QStringList` and other `QVariant`-compatible types round-trip automatically — no manual comma-split/join.

Resolved pattern — centralized loader avoiding string-key typos:

```
struct MonitorConfig {
    QString brokerHost; int brokerPort; bool useTls;
    QString serialDevicePath; int baudRate;
    double tempHighThreshold; QStringList sensorIds;
};

class ConfigLoader {
public:
    static MonitorConfig load(const QString &path)
    {
        QSettings settings(path, QSettings::IniFormat);
        MonitorConfig cfg;
        settings.beginGroup("mqtt");
        cfg.brokerHost = settings.value("broker_host", "localhost").toString();
        cfg.brokerPort = settings.value("broker_port", 1883).toInt();
        settings.endGroup();
        // ... serial, thresholds groups similarly
        return cfg;
    }
};
```

Exactly one function touches raw key strings; everywhere else uses compiler-checked struct fields — converting a class of silent string-typo bugs into compile-time-checked access.

Key takeaways - `QSettings` gives typed hierarchical config via `QVariant` — no manual type conversion. - Use `IniFormat` with an explicit path for deployed services. - `value(key, default)` — always supply a sensible default. - Centralize all `QSettings` reads into one loader producing a plain struct.

Day 10: QJsonDocument / QJsonObject / QJsonArray

Theory: `QJsonObject` preserves insertion order (unlike `QMap`). Every JSON number is stored as a `double` internally — exact integer fidelity is only guaranteed up to 2^{53} . `QJsonValue::toVariant() / fromVariant()` bridge to `QVariant`.

Resolved example 1 — building, serializing, the int/double gotcha:

```
QJsonObject reading;
reading["device_id"] = "sensor-07";
reading["temperature"] = 23.5;
reading["humidity"] = 61;
reading["reading_count"] = 9007199254740993LL;    // beyond 2^53

QJsonDocument doc(reading);
QByteArray compact = doc.toJson(QJsonDocument::Compact);    // for network payloads
QByteArray indented = doc.toJson(QJsonDocument::Indented);    // for debug output

qDebug() << reading["humidity"].type();    // QJsonValue::Double, NOT Int
qDebug() << reading["reading_count"].toVariant().toLongLong();
// 9007199254740992 -- off by one: precision loss beyond 2^53
```

For exact integer fidelity beyond 2^{53} (nanosecond timestamps, hashes), encode as a string instead of relying on JSON's native number type.

Resolved example 2 — parsing untrusted input, proper error handling:

```
void handleIncomingPayload(const QByteArray &payload)
{
    QJsonParseError parseError;
    QJsonDocument doc = QJsonDocument::fromJson(payload, &parseError);
    if (parseError.error != QJsonParseError::NoError) {
        qWarning() << "JSON parse failed:" << parseError.errorString(); return;
    }
    if (!doc.isObject()) { qWarning() << "Expected object"; return; }
    QJsonObject obj = doc.object();

    if (!obj.contains("device_id") || !obj["device_id"].isString()) {
        qWarning() << "Missing or invalid 'device_id'"; return;
    }
    if (!obj.contains("temperature") || !obj["temperature"].isDouble()) {
        qWarning() << "Missing or invalid 'temperature'"; return;
    }
    qDebug() << "Parsed OK";
}
```

Three distinct failure modes — malformed syntax, wrong field type, missing field — each need their own check; a naive `toDouble()` call without validation can't distinguish a parsing bug from a genuinely zero reading.

Resolved example 3 — QJsonArray and the QVariant bridge:

```
QJsonArray batch;  
batch.append(QJsonObject{{"device_id", "sensor-01"}, {"temperature", 23.5}});  
// ... more entries  
  
double total = 0;  
for (const QJsonValue &val : batch) total += val.toObject()["temperature"].toDouble();  
  
QVariant asVariant = batch.toVariantList(); // bridges into QSettings/setProperty world
```

Key takeaways - `QJsonObject` preserves insertion order; all numbers are doubles internally (2^{53} precision limit). - Always check `QJsonParseError` after `fromJson()`. - Validate both key existence and value type before extracting. - `toVariant()/toVariantList()` bridge JSON into the `QVariant` world (`QSettings`, `QObject` properties).

Day 11: QRegularExpression — Parsing Serial Line Formats Properly

Theory: `QRegularExpression` (PCRE2-based) supports named capture groups — `.captured("name")` instead of positional indices, self-documenting and immune to index-shift bugs.

Resolved example 1 — named captures, anchored:

```
QRegularExpression re(R"(^SENSOR:(?<id>\d+):TEMP:(?<temp>-\d+\.\d*):TS:(?<ts>\d+)\$)");

QRegularExpressionMatch match = re.match(line);
if (match.hasMatch()) {
    QString id = match.captured("id");
    double temp = match.captured("temp").toDouble();
    qint64 ts = match.captured("ts").toLongLong();
}
```

Anchoring (`^...$`) rejects “looks almost right but has trailing garbage” lines — without it, a regex could match a substring buried inside noise, silently accepting corrupted input.

Resolved example 2 — `globalMatch` for concatenated readings, compiled once:

```
static const QRegularExpression re(R"((?<type>TEMP|HUMIDITY):(?<value>-\d+\.\d*))");
// static const: compiled ONCE, reused across every call -- avoid recompiling in a hot loop

QRegularExpressionMatchIterator it = re.globalMatch(blob);
while (it.hasNext()) {
    QRegularExpressionMatch m = it.next();
    // process m.captured("type"), m.captured("value")
}
```

Resolved example 3 — validation with specific diagnostics:

```

bool validateDeviceId(const QString &id, QString *errorOut = nullptr)
{
    static const QRegularExpression re(R"^sensor-\\d{2}$");
    if (re.match(id).hasMatch()) return true;
    if (errorOut) {
        if (id.length() != 9) *errorOut = QString("expected 9 chars, got
%1").arg(id.length());
        else if (!id.startsWith("sensor-")) *errorOut = "must start with 'sensor-'";
        else *errorOut = "trailing part must be exactly 2 digits";
    }
    return false;
}

```

For human-facing validation, diagnosing *why* is worth the extra code; for machine-generated protocol parsing, a flat reject-and-skip suffices.

Key takeaways - Always anchor a regex meant to validate an entire line. - Named capture groups are self-documenting and index-shift-proof. - Construct `QRegularExpression` once (`static const`) and reuse — recompiling per call wastes work in a hot path. - `globalMatch()` + iterator extracts multiple matches from one buffered blob.

Day 12: Q_PROPERTY in Depth & Custom QObject-Derived Data Model Classes

Theory: `Q_PROPERTY(Type name READ getter WRITE setter NOTIFY signal)` registers a named property with moc, generating real signal/slot integration and enabling generic `property() / setProperty()` access by string name — routed through your actual accessor logic.

Resolved example — a data model class with NOTIFY and no-op guards:

```
class DeviceReading : public QObject
{
    Q_OBJECT
    Q_PROPERTY(QString deviceId READ deviceId WRITE setDeviceId NOTIFY deviceIdChanged)
    Q_PROPERTY(double temperature READ temperature WRITE setTemperature NOTIFY
temperatureChanged)
    Q_PROPERTY(QDateTime timestamp READ timestamp WRITE setTimestamp NOTIFY timestampChanged)
    Q_PROPERTY(bool online READ isOnline WRITE setOnline NOTIFY onlineChanged)
public:
    explicit DeviceReading(QObject *parent = nullptr) : QObject(parent) {}

    QString deviceId() const { return m_deviceId; }
    void setDeviceId(const QString &id)
    {
        if (m_deviceId == id) return; // no-op guard: don't emit for a non-change
        m_deviceId = id;
        emit deviceIdChanged(m_deviceId);
    }
    double temperature() const { return m_temperature; }
    void setTemperature(double t)
    {
        if (qFuzzyCompare(m_temperature, t)) return;
        m_temperature = t;
        emit temperatureChanged(m_temperature);
    }
    QDateTime timestamp() const { return m_timestamp; }
    void setTimestamp(const QDateTime &ts) { if (m_timestamp == ts) return; m_timestamp =
ts; emit timestampChanged(m_timestamp); }
    bool isOnline() const { return m_online; }
    void setOnline(bool o) { if (m_online == o) return; m_online = o; emit
onlineChanged(m_online); }

signals:
    void deviceIdChanged(const QString&);
    void temperatureChanged(double);
    void timestampChanged(const QDateTime&);
    void onlineChanged(bool);

private:
    QString m_deviceId; double m_temperature = 0.0; QDateTime m_timestamp; bool m_online =
false;
};
```

Generic access by string name, with zero compile-time knowledge of `DeviceReading`, still routes through the real setter (including the no-op guard):

```
reading.setProperty("temperature", 30.0); // triggers the real temperatureChanged signal
```

Why the no-op guard matters concretely: without it, 1000 identical writes to the same value fire 1000 redundant NOTIFY signals — if connected to something like “publish to MQTT,” that’s 1000x unnecessary work for a sensor that hasn’t changed. The guard costs one comparison per setter call.

Key takeaways - `Q_PROPERTY` + `NOTIFY` generates real signal/slot integration, not just documentation. - Always guard setters with an equality check before mutating/emitting. - `property() / setProperty()` route through the real accessor logic, including guards. - Prefer a typed `QObject`-derived model over loose `QJsonObject / QVariantMap` when data needs to be observed, validated, or accessed generically.

Day 13: QDateTime & QTimeZone — Timestamping Correctly

Theory: the UTC-vs-local trap

`QDateTime::currentDateTime()` returns local time — ambiguous across a multi-timezone deployment and across DST transitions (a repeated or skipped hour once a year each way). **Always store timestamps as UTC** (`currentDateTimeUtc()`); convert to local only for display.

Resolved example — the untagged-constructor bug, made concrete:

```
QDateTime reading(QDate(2026,7,22), QTime(14,45,12,345), QTimeZone::UTC);
QDebug() << reading.toString(Qt::ISODateWithMs); // correctly appends 'Z'

QDateTime untagged(QDate(2026,7,22), QTime(14,45,12,345)); // NO timezone argument
QDebug() << untagged.timeSpec(); // Qt::LocalTime -- not what was meant!
QDebug() << untagged.toUTC().toString(Qt::ISODateWithMs);
// WRONG by 3 hours if 14:45:12 was meant to BE the UTC instant --
// Qt silently assumed local time (EEST, UTC+3) and "corrected" it.
```

Constructing a `QDateTime` without an explicit `QTimeZone` defaults to `Qt::LocalTime` — a common, silent bug source invisible until a device runs in a different timezone or a DST boundary is crossed.

Resolved usage in DeviceReading:

```
reading.setTimestamp(QDateTime::currentDateTimeUtc()); // always UTC, always comparable
```

Key takeaways - Always use `currentDateTimeUtc()` for anything stored or transmitted; convert to local only at display time. - DST transitions make local time genuinely ambiguous or invalid once a year each way — a structural reason UTC is the only safe storage representation. - An untagged `QDateTime` constructor defaults to `Qt::LocalTime` — pass `QTimeZone::UTC` explicitly when that's the intent. - `Qt::ISODateWithMs` only correctly appends timezone info if the `QDateTime` is properly tagged at construction.

Day 14: Mini-Project — Serial-Line Parser → Structured JSON Record Pipeline

Combines Days 8-13: parse (regex) → validate/model (Q_PROPERTY class) → UTC-timestamp → serialize (JSON) → ready for storage or transmission.

```
// seriallineparser.h
struct ParsedFields { QString deviceId; double temperature; };

class SerialLineParser
{
public:
    SerialLineParser() : m_pattern(R"(^SENSOR:(?<id>\d+):TEMP:(?<temp>-?\d+\.\d*)$)") {}
    std::optional<ParsedFields> parse(const QString &rawLine) const
    {
        QRegularExpressionMatch match = m_pattern.match(rawLine);
        if (!match.hasMatch()) return std::nullopt;
        ParsedFields fields;
        fields.deviceId = "sensor-" + match.captured("id");
        fields.temperature = match.captured("temp").toDouble();
        return fields;
    }
private:
    QRegularExpression m_pattern;
};
```

```
// readingserializer.h
class ReadingSerializer
{
public:
    static QJsonObject toJson(const DeviceReading &reading)
    {
        QJsonObject obj;
        obj["device_id"] = reading.deviceId();
        obj["temperature"] = reading.temperature();
        // defensive: re-assert UTC at THIS single boundary regardless of upstream
        correctness obj["timestamp_utc"] = reading.timestamp().toUTC().toString(Qt::ISODateWithMs) + "Z";
        obj["online"] = reading.isOnline();
        return obj;
    }
};
```

Driver code runs a batch of raw lines (some malformed) through parse → model → serialize, using `std::optional` to force the caller to handle rejection before touching a possibly-empty result. Output: 3 accepted, 3 rejected (garbage, non-numeric temp, unexpected trailing data — the last correctly caught by the anchored regex).

Key takeaways - A real ingestion pipeline composes single-responsibility stages: parse → validate/model → timestamp → serialize → persist/transmit. - `std::optional` for “parse might fail” composes more safely than `bool+out-param`. - Anchored regexes correctly reject “almost right but has trailing garbage” — a realistic corruption mode. - Put defensive correctness (like re-asserting UTC) at the single boundary where data leaves the system, rather than trusting every upstream call site. ## Day 15: Qt’s Threading Model — Thread Affinity and Per-Thread Event Loops

Theory: thread affinity

Every `QObject` has a thread affinity (`QObject::thread()`) — set at construction to the constructing thread, changeable exactly once safely via `moveToThread()`. Affinity determines where a queued slot invocation actually executes: a queued signal posts an event to the *receiver’s* thread’s event queue, dispatched by that thread’s own event loop. Every thread that wants queued signals or `QTimer` must run its own event loop (`exec()`).

Resolved example — a `QThread` with NO event loop cannot run a `QTimer`:

```
class BadWorkerThread : public QThread
{
protected:
    void run() override // deliberately WRONG: subclassing QThread, no exec()
    {
        QTimer timer;
        QObject::connect(&timer, &QTimer::timeout, [] {
            qDebug() << "this will NEVER print -- no event loop to dispatch it";
        });
        timer.start(500);
        QThread::sleep(2); // only safe here because there's no event loop to block
    }
};
```

The timer never fires — `run()` never calls `exec()`, so there’s no event loop on that thread to notice the timer expired.

Resolved example — a `QThread` WITH an event loop (default, unmodified `run()`) works correctly:

```

class GoodWorker : public QObject
{
    Q_OBJECT
public slots:
    void startTicking()
    {
        auto *timer = new QTimer(this);
        connect(timer, &QTimer::timeout, this, &GoodWorker::onTick);
        timer->start(500);
    }
signals:
    void tickOccurred(int count);
private slots:
    void onTick() { /* emit tickOccurred(...) */ }
};

QThread workerThread;           // default run() just calls exec()
GoodWorker worker;
worker.moveToThread(&workerThread); // reassigns thread affinity

QObject::connect(&workerThread, &QThread::started, &worker, &GoodWorker::startTicking);
QObject::connect(&worker, &GoodWorker::tickOccurred, [](int c) {
    qDebug() << "observer on thread:" << QThread::currentThread(); // MAIN thread
});
workerThread.start();

```

The `tickOccurred` signal is emitted on the worker thread, but the connected main-thread lambda executes on the main thread automatically — an auto-detected queued connection, delivered through the main thread's own event loop.

Key takeaways - Thread affinity is set at construction, changeable via `moveToThread()`. - A thread only has a usable event loop if something calls `exec()` on it — `QThread`'s default `run()` does exactly this. - Without an event loop, a thread's `QObject`s can't use `QTimer` or receive queued slot invocations. - The idiomatic pattern: don't subclass `QThread`; move a plain `QObject` worker onto a plain `QThread` instance instead.

Day 16: The QThread + moveToThread Worker Pattern, Fully Formalized

Theory: `QThread` is a lifecycle handle (start/quit/wait); the worker is a separate `QObject` moved onto it. Do initialization that must run on the new thread in a slot connected to `QThread::started`, not in the worker's constructor (which runs before the move, on the original thread).

Resolved example — full lifecycle:

```
class SerialWorker : public QObject
{
    Q_OBJECT
public slots:
    void initialize() // connected to QThread::started
    {
        auto *pollTimer = new QTimer(this);
        connect(pollTimer, &QTimer::timeout, this, &SerialWorker::simulateRead);
        pollTimer->start(300);
    }
    void shutdown() { emit finished(); }
signals:
    void lineReady(const QString &line);
    void finished();
private slots:
    void simulateRead() { emit lineReady(/* fake line */); }
};

// main.cpp
auto *workerThread = new QThread();
auto *worker = new SerialWorker(); // NO parent -- required before moveToThread
worker->moveToThread(workerThread);

QObject::connect(workerThread, &QThread::started, worker, &SerialWorker::initialize);
QObject::connect(worker, &SerialWorker::finished, workerThread, &QThread::quit);
QObject::connect(workerThread, &QThread::finished, worker, &QObject::deleteLater);
QObject::connect(workerThread, &QThread::finished, workerThread, &QObject::deleteLater);

// requesting shutdown from the main thread MUST go through invokeMethod:
QMetaObject::invokeMethod(worker, "shutdown", Qt::QueuedConnection);
// NEVER worker->shutdown() directly -- that runs on the CALLER's thread, not worker's.

workerThread->start();
```

Resolved parenting trap:

```
auto *worker = new SerialWorker(someOwner); // parented BEFORE moving
worker->moveToThread(workerThread);
// Qt WARNS at runtime: "QObject::moveToThread: Cannot move objects with a parent"
// The move is refused -- worker stays on the original thread.
```

Never give a to-be-moved object a parent beforehand; manage lifetime via `deleteLater()` connected to `QThread::finished` instead.

Key takeaways - `QThread` and the worker are separate objects with separate responsibilities. - Initialize on the new thread via a slot connected to `started`, not the constructor. - An object with a parent cannot be `moveToThread()`'d — Qt refuses and warns. - Call methods on a worker via `QMetaObject::invokeMethod(obj, "method", Qt::QueuedConnection)`, never directly. - Standard shutdown chain: `finished()` → `quit()` → `finished()` → `deleteLater()` ×2.

Day 17: QMutex & QReadWriteLock — Shared Data Between Threads

Theory: if two threads never touch the same variable directly, no mutex is needed — signal/slot argument-copying already avoids shared memory. A mutex is needed only when multiple threads must read/write the *same* in-memory structure directly.

- `QMutex` — exclusive lock, one thread at a time.
- `QReadWriteLock` — many simultaneous readers, exclusive writer. Correct when reads vastly outnumber writes.

Resolved example — the race, then the fix:

```
// BROKEN: unprotected shared counter
int g_unprotectedCounter = 0;
void incrementManyTimes() { for (int i=0;i<100000;++i) ++g_unprotectedCounter; }
// two threads -> result is nondeterministic, usually LESS than 200000 (lost updates)

// FIXED: QMutexLocker (RAII, like lock_guard)
QMutex g_counterMutex;
int g_protectedCounter = 0;
void incrementSafely() {
    for (int i=0;i<100000;++i) {
        QMutexLocker locker(&g_counterMutex);
        ++g_protectedCounter;
    }
}
// result: exactly 200000, every run
```

Resolved example — QReadWriteLock for a read-heavy shared cache:

```
class SharedConfigCache
{
public:
    QVariant get(const QString &key) const
    { QReadLocker locker(&m_lock); return m_data.value(key); }
    void set(const QString &key, const QVariant &value)
    { QWriteLocker locker(&m_lock); m_data[key] = value; }
private:
    mutable QReadWriteLock m_lock;
    QMap<QString, QVariant> m_data;
};
```

Multiple readers proceed concurrently without blocking each other; only a write briefly excludes everyone. A plain `QMutex` would needlessly serialize the readers too.

Key takeaways - Reach for a mutex only when threads share the *same* in-memory structure directly. - Always use RAII lockers (`QMutexLocker`, `QReadLocker`, `QWriteLocker`) rather than manual lock/unlock. - `QReadWriteLock` beats `QMutex` when reads vastly outnumber writes and don't need to block each other. - An unprotected read-modify-write produces a genuine, nondeterministic race — a different wrong answer each run.

Day 18: QThreadPool & QRunnable — Task-Based Concurrency

Theory: `QThreadPool + QRunnable` is for many short-lived, independent tasks (unlike Day 16's long-lived worker). The pool reuses a bounded set of threads (default `idealThreadCount()`) rather than spawning one OS thread per task.

Resolved example — parallel parsing with thread-safe result collection:

```
class ParseTask : public QRunnable
{
public:
    ParseTask(const QString &line, QList<DeviceReading*> *results, QMutex *mutex)
        : m_rawLine(line), m_results(results), m_resultsMutex(mutex) {}
    // setAutoDelete(true) is the default -- pool deletes this task after run()

    void run() override
    {
        static thread_local SerialLineParser parser; // one parser per thread, no shared
state
        auto parsed = parser.parse(m_rawLine);
        if (!parsed) return;
        auto *reading = new DeviceReading();
        reading->setDeviceId(parsed->deviceId);
        reading->setTemperature(parsed->temperature);
        {
            QMutexLocker locker(m_resultsMutex); // protecting the SHARED list
            m_results->append(reading);
        }
    }
private:
    QString m_rawLine;
    QList<DeviceReading*> *m_results;
    QMutex *m_resultsMutex;
};

// usage:
QThreadPool *pool = QThreadPool::globalInstance();
for (const QString &line : batch) pool->start(new ParseTask(line, &results, &resultsMutex));
pool->waitForDone();
```

Resolved: over-subscription doesn't help CPU-bound work. Running the same batch with `maxThreadCount = cores` vs `cores * 4` gives roughly equal (or slightly worse) time — more threads than physical cores just adds scheduling overhead.

Resolved: QRunnable + QObject for completion signaling (QRunnable alone has no built-in completion notification):

```

class SignalingTask : public QObject, public QRunnable
{
    Q_OBJECT
public:
    explicit SignalingTask(int input) : m_input(input) {}
    void run() override { emit finished(m_input, m_input * m_input); }
signals:
    void finished(int input, int result);
private:
    int m_input;
};
// task->setAutoDelete(false); connect to finished(), delete task manually inside the slot

```

Completion order from a pool is inherently unordered — track and reorder explicitly if the use case requires it.

Key takeaways - Use `QThreadPool/QRunnable` for many short-lived tasks; `QThread + moveToThread` for one long-lived worker. - Default `setAutoDelete(true)` means the pool deletes your task — set `false` only with a clear alternative ownership plan. - Over-subscribing threads beyond core count doesn't help CPU-bound work. - `QRunnable` has no built-in completion signal; combine with `QObject` when one is needed.

Day 19: QProcess — Spawning and Communicating with External Processes

Theory: `QProcess` is a `QIODevice` wrapping `stdin/stdout/stderr`. Starting a process is non-blocking; results arrive via `readyReadStandardOutput()`, `errorOccurred()`, `finished()` through the event loop.

Resolved example — async run, proving non-blocking start:

```
auto *process = new QProcess(&app);
connect(process, &QProcess::readyReadStandardOutput, [process]() {
    qDebug() << "[stdout]" << process->readAllStandardOutput().trimmed();
});
connect(process, &QProcess::finished, [&app](int code, QProcess::ExitStatus status) {
    qDebug() << "finished, code:" << code;
    app.quit();
});
connect(process, &QProcess::errorOccurred, [](QProcess::ProcessError e) {
    qWarning() << "Process error:" << e;
});

qDebug() << "Starting...";
process->start("ls", QStringList{"-la", "/tmp"}); // non-blocking, returns immediately
qDebug() << "start() returned"; // prints BEFORE any output arrives
```

Resolved: piping stdin, and the hang trap. A process reading until EOF (like `cat`) hangs forever unless you call `closeWriteChannel()` after writing — this signals EOF on `stdin`, letting the process terminate.

```
process->write("TEMP:23.5\nHUMIDITY:61\n");
process->closeWriteChannel(); // without this, 'cat' waits on stdin forever
```

`errorOccurred(QProcess::FailedToStart)` fires for a nonexistent executable — such a process never reaches `finished()` at all, so check errors explicitly rather than assuming `finished()` will always fire.

Resolved: the one legitimate synchronous case — a bounded-timeout, one-off startup check *before* `app.exec()` :

```
if (!process.waitForStarted(2000)) { /* handle */ }
if (!process.waitForFinished(3000)) { process.kill(); /* handle */ }
```

Acceptable only because it's bounded, happens before the event loop is running, and is a one-off check, not a repeated hot-path pattern.

Key takeaways - `QProcess` is a `QIODevice`; starting is non-blocking, results delivered via signals. - Writing to a process's stdin and expecting termination requires `closeWriteChannel()` — a common silent-hang trap. - A failed-to-start process never reaches `finished()` — check `errorOccurred()` explicitly. - Bounded synchronous waits are acceptable only for brief, justified, pre-event-loop checks.

Day 20: QFileSystemWatcher — Reacting Live to File/Config Changes

Theory: uses OS-level notification (inotify on Linux) instead of polling — lower latency, no wasted work, delivered as a signal through the event loop.

Two real caveats: 1. Atomic-save editors (vim, most GUI editors) rename a temp file over the original — the watcher can silently drop the path from its watch list. Fix: re-add the path unconditionally inside `fileChanged()`. 2. A single logical save can fire `fileChanged()` multiple times. Fix: debounce with a short `QTimer`.

Resolved example:

```
class ConfigWatcher : public QObject
{
    Q_OBJECT
public:
    explicit ConfigWatcher(const QString &path, QObject *parent = nullptr)
        : QObject(parent), m_path(path)
    {
        m_watcher.addPath(m_path);
        connect(&m_watcher, &QFileSystemWatcher::fileChanged, this,
            &ConfigWatcher::onFileChanged);
        m_debounce.setSingleShot(true);
        m_debounce.setInterval(300);
        connect(&m_debounce, &QTimer::timeout, this, &ConfigWatcher::reloadConfig);
        reloadConfig();
    }
private slots:
    void onFileChanged(const QString &path)
    {
        if (!m_watcher.files().contains(m_path)) {
            m_watcher.addPath(m_path); // re-add unconditionally: fixes caveat 1
        }
        m_debounce.start(); // restart, not fire immediately: fixes caveat 2
    }
    void reloadConfig() { /* MonitorConfig cfg = ConfigLoader::load(m_path); emit ... */ }
private:
    QString m_path;
    QFileSystemWatcher m_watcher;
    QTimer m_debounce;
};
```

Without the re-add fix, the first edit works once, then silently stops detecting further changes — a common, easy-to-miss production bug.

Resolved: watching a directory — `directoryChanged()` says *that* something changed, not *what*; diff against a known file-list snapshot to find new/removed files.

Key takeaways - Event-driven file watching (inotify-backed) beats polling for latency and efficiency. - Always re-add a watched path unconditionally after `fileChanged()` — atomic-save editors are common, not an edge case. - Debounce with a short `QTimer` to collapse multiple signals from one logical save. - `directoryChanged()` requires diffing a known snapshot to determine what specifically changed.

Day 21: Mini-Project — Multi-Threaded Worker Pool Processing Simulated Sensor Readings

Synthesizes Days 15–20: `QThreadPool/QRunnable` for parsing, `QMutex` for a shared result list, `QAtomicInt` for a lock-free completion counter, `QReadWriteLock` for a live-reloadable, read-heavy threshold.

```
class ReadingBatchProcessor : public QObject
{
    Q_OBJECT
public:
    double highTempThreshold() const { QReadLocker l(&m_thresholdLock); return
m_highTempThreshold; }
    void setHighTempThreshold(double t) { QWriteLocker l(&m_thresholdLock);
m_highTempThreshold = t; }

    void processBatch(const QStringList &rawLines)
    {
        m_expectedCount = rawLines.size();
        m_completedCount.storeRelaxed(0);
        m_results.clear();
        QThreadPool *pool = QThreadPool::globalInstance();
        for (const QString &line : rawLines) pool->start(new ParseTask(line, this));
    }

    // called by each ParseTask, on a WORKER thread
    void reportResult(DeviceReading *reading)
    {
        if (reading) { QMutexLocker l(&m_resultsMutex); m_results.append(reading); }
        else m_rejectedCount.fetchAndAddRelaxed(1);

        int done = m_completedCount.fetchAndAddRelaxed(1) + 1;
        if (done == m_expectedCount)
            emit batchCompleted(m_results.size(), m_rejectedCount.loadRelaxed());
        // "last task reports completion" -- avoids any blocking waitForDone() call
    }
signals:
    void batchCompleted(int successCount, int rejectedCount);
private:
    QMutex m_resultsMutex;
    QList<DeviceReading*> m_results;
    QAtomicInt m_completedCount{0}, m_rejectedCount{0};
    int m_expectedCount = 0;
    mutable QReadWriteLock m_thresholdLock;
    double m_highTempThreshold = 30.0;
};
```

A main-thread `QTimer::singleShot(0, ...)` deliberately lowers the threshold mid-batch, racing against worker threads reading it — exactly the scenario `QReadWriteLock` exists for, made visible rather than left to chance. `batchCompleted`

is emitted from whichever worker thread finishes last, but the connected main-thread lambda still executes on the main thread — auto-detected queued delivery (Day 15).

Key takeaways - A single component can legitimately combine `QThreadPool`, `QMutex`, `QAtomicInt`, and `QReadWriteLock`, each matched to its actual access pattern rather than one blanket mutex. - “Last task reports completion” keeps the main thread’s event loop responsive instead of blocking on `waitForDone()`. - Live-reconfiguring shared state while workers actively read it is a real requirement, not an academic exercise — exactly why Day 17’s lock disciplines exist. ## Day 22: `QTcpSocket` & `QTcpServer` — Async Client-Server Networking

Theory: `QTcpSocket` is a `QIODevice` with an async connection lifecycle expressed as signals: `connected()`, `readyRead()`, `disconnected()`, `errorOccurred()`. `QTcpServer` emits `newConnection()`; you call `nextPendingConnection()` to get each client’s socket.

Resolved example — echo server, correctly draining pending connections:

```
class EchoServer : public QObject
{
    Q_OBJECT
public:
    explicit EchoServer(quint16 port, QObject *parent = nullptr) : QObject(parent)
    {
        connect(&m_server, &QTcpServer::newConnection, this, &EchoServer::onNewConnection);
        m_server.listen(QHostAddress::Any, port);
    }
private slots:
    void onNewConnection()
    {
        while (m_server.hasPendingConnections()) { // loop -- don't assume exactly one per
signal
            QTcpSocket *client = m_server.nextPendingConnection();
            client->setParent(this); // parent to server (Day 5) -- cascades cleanup
            connect(client, &QTcpSocket::readyRead, this, [this, client]() {
onClientDataReady(client); });
            connect(client, &QTcpSocket::disconnected, this, [client]() { client->
deleteLater(); });
        }
    }
    void onClientDataReady(QTcpSocket *client)
    {
        QByteArray data = client->readAll();
        client->write("ECHO: " + data);
    }
private:
    QTcpServer m_server;
};
```

Resolved example — client, with essential error handling:


```

auto *socket = new QTcpSocket(&app);
connect(socket, &QTcpSocket::connected, [socket]() { socket->write("TEMP:24.1\n"); });
connect(socket, &QTcpSocket::readyRead, [socket]() { qDebug() << socket->readAll(); socket-
>disconnectFromHost(); });
connect(socket, &QTcpSocket::errorOccurred, [&app](QAbstractSocket::SocketError e) {
    qDebug() << "Socket error:" << e;    // WITHOUT this, a failed connect() never fires
connected()
    app.quit();                          // and the program silently hangs forever
});
socket->connectToHost("localhost", 9999);

```

Resolved: TCP has no message boundaries. `readyRead()` means “more bytes arrived,” not “one complete message arrived.” The fix is length-prefix framing with a buffer that accumulates partial data:

```

class FramedReader : public QObject
{
    Q_OBJECT
public slots:
    void onReadyRead()
    {
        m_buffer.append(m_socket->readAll());
        while (true) {
            if (m_expectedLength == 0) {
                if (m_buffer.size() < 4) return;    // header not fully arrived yet
                QDataStream hs(m_buffer.left(4));
                hs.setVersion(QDataStream::Qt_6_5);
                hs >> m_expectedLength;
                m_buffer.remove(0, 4);
            }
            if (static_cast<quint32>(m_buffer.size()) < m_expectedLength) return;    //
partial message

            QByteArray messageData = m_buffer.left(m_expectedLength);
            m_buffer.remove(0, m_expectedLength);
            m_expectedLength = 0;
            QJsonDocument doc = QJsonDocument::fromJson(messageData);
            if (doc.isObject()) emit messageReady(doc.object());
            // loop continues -- another full message might already be buffered
        }
    }
signals:
    void messageReady(const QJsonObject &json);
private:
    QTcpSocket *m_socket;
    QByteArray m_buffer;
    quint32 m_expectedLength = 0;
};

```

Key takeaways - Always wire `errorOccurred()` — a failed connection silently never reaches `connected()`. - Loop with `hasPendingConnections()/nextPendingConnection()`, not a single assumed connection per signal. - TCP is a byte stream, not a message stream — any protocol over raw TCP needs explicit framing and a partial-data buffer.

Day 23: QUdpSocket — Connectionless, Datagram-Based Communication

Theory: each `writeDatagram()` / `readyRead()` corresponds to one discrete packet — datagrams preserve message boundaries natively, unlike TCP. Trade-off: no delivery guarantee, no ordering, no retransmission — all must be built at the application level if needed.

Resolved example — telemetry sender/receiver with sequence-number gap detection:

```

class TelemetrySender : public QObject
{
    Q_OBJECT
private slots:
    void sendReading()
    {
        QJsonObject reading;
        reading["device_id"] = "sensor-07";
        reading["seq"] = ++m_seq;    // lets receiver detect drops/reordering
        QByteArray datagram = QJsonDocument(reading).toJson(QJsonDocument::Compact);
        m_socket.writeDatagram(datagram, QHostAddress(m_host), m_port);    // ONE call = ONE
packet
    }
private:
    QUdpSocket m_socket; QString m_host; quint16 m_port; int m_seq = 0;
};

class TelemetryReceiver : public QObject
{
    Q_OBJECT
private slots:
    void onReadyRead()
    {
        while (m_socket.hasPendingDatagrams()) {    // drain ALL pending, like Day 22's TCP
loop
            QByteArray datagram; datagram.resize(m_socket.pendingDatagramSize());
            QHostAddress sender; quint16 senderPort;
            m_socket.readDatagram(datagram.data(), datagram.size(), &sender, &senderPort);
            QJsonDocument doc = QJsonDocument::fromJson(datagram);
            if (!doc.isObject()) { qWarning() << "malformed datagram";
continue; }    // skip, don't stall
            int seq = doc.object()["seq"].toInt();
            if (m_lastSeq != -1 && seq != m_lastSeq + 1)
                qWarning() << "Sequence gap/reorder: expected" << (m_lastSeq+1) << "got" <<
seq;
            m_lastSeq = seq;
        }
    }
private:
    QUdpSocket m_socket; int m_lastSeq = -1;
};

```

Resolved: broadcast discovery — `socket.writeDatagram(data, QHostAddress::Broadcast, port)` reaches every listener on the subnet with no per-recipient connection, useful for device discovery announcements with no TCP equivalent.

Key takeaways - UDP preserves message boundaries natively — no framing buffer needed, unlike TCP. - No ordering/delivery/retransmission guarantees — build sequence numbers or acks yourself if needed. - Loop with `hasPendingDatagrams()`, matching Day 22's connection-draining discipline. - Choose UDP for high-frequency, loss-tolerant telemetry and discovery; TCP for guaranteed ordered delivery.

Day 24: QNetworkAccessManager — HTTP Requests

Theory: `QNetworkAccessManager` (QNAM) handles HTTP/HTTPS protocol details above raw sockets. Requests return a `QNetworkReply` (itself a `QIODevice`) immediately; read the body once `finished()` fires. **Construct one QNAM for your app's lifetime** — it manages connection pooling/keep-alive internally.

Resolved example — GET with proper status/error handling:

```
auto *manager = new QNetworkAccessManager(&app); // ONE instance, reused
QNetworkRequest request(QUrl("https://httpbin.org/json"));
QNetworkReply *reply = manager->get(request);

connect(reply, &QNetworkReply::finished, [reply, &app]() {
    if (reply->error() != QNetworkReply::NoError)
    { // finished() fires on BOTH success and failure
        qWarning() << "Request failed:" << reply->errorString();
        reply->deleteLater(); app.quit(); return;
    }
    int status = reply->attribute(QNetworkRequest::HttpStatusCodeAttribute).toInt();
    if (status >= 200 && status < 300) {
        QJsonDocument doc = QJsonDocument::fromJson(reply->readAll());
        // ... use doc
    }
    reply->deleteLater(); // NEVER delete reply directly inside its own finished() slot
    app.quit();
});
```

Resolved example — POST with JSON body:

```
QNetworkRequest request(QUrl("https://httpbin.org/post"));
request.setHeader(QNetworkRequest::ContentTypeHeader, "application/json"); // must be set explicitly
QNetworkReply *reply = manager->post(request, jsonBody);
```

Resolved: timeouts. A stuck/unreachable request has no guaranteed built-in timeout on all Qt6 versions:

```
request.setTransferTimeout(3000); // Qt 6.7+
// backstop for portability across Qt6 versions:
QTimer *guard = new QTimer(&app);
guard->setSingleShot(true);
connect(guard, &QTimer::timeout, [reply]() { if (reply->isRunning()) reply->abort(); });
guard->start(5000);
```

Key takeaways - Construct one `QNetworkAccessManager` for the app's lifetime — never one per request. - `finished()` fires on both success and failure — always check `error()` before reading the body. - Never `delete reply` inside its own

`finished()` slot — always `deleteLater()` . - Set `Content-Type` explicitly for a JSON POST body; back up `setTransferTimeout()` with a manual guard timer for portability.

Day 25: QStateMachine — Formal State Machines for Connection Lifecycles

Theory: a hand-rolled enum+switch degrades as complexity grows — every handler must independently guard against invalid transitions. `QStateMachine` makes valid transitions explicit, declared edges; an event with no registered transition from the current state is structurally a no-op.

Resolved example — device connection lifecycle:

```
class DeviceConnection : public QObject
{
    Q_OBJECT
public:
    explicit DeviceConnection(QObject *parent = nullptr) : QObject(parent) { setupStates(); }
    void start() { m_machine.start(); }
signals:
    void connectRequested(); void connectionEstablished();
    void connectionFailed(); void disconnectedUnexpectedly(); void retryRequested();
private:
    void setupStates()
    {
        m_disconnected = new QState(&m_machine);
        m_connecting = new QState(&m_machine);
        m_connected = new QState(&m_machine);
        m_reconnecting = new QState(&m_machine);

        connect(m_connected, &QState::entered, [this] { m_retryCount = 0; });
        connect(m_reconnecting, &QState::entered, [this] {
            ++m_retryCount;
            if (m_retryCount > 3) return; // give up automatic retry past a cap
            QTimer::singleShot(1000, this, [this]{ emit retryRequested(); });
        });

        // declarative, VALID-ONLY-FROM-THIS-STATE transitions:
        m_disconnected->addTransition(this, &DeviceConnection::connectRequested,
m_connecting);
        m_connecting->addTransition(this, &DeviceConnection::connectionEstablished,
m_connected);
        m_connecting->addTransition(this, &DeviceConnection::connectionFailed,
m_reconnecting);
        m_connected->addTransition(this, &DeviceConnection::disconnectedUnexpectedly,
m_reconnecting);
        m_reconnecting->addTransition(this, &DeviceConnection::retryRequested, m_connecting);

        m_machine.setInitialState(m_disconnected);
    }
    QStateMachine m_machine;
    QState *m_disconnected, *m_connecting, *m_connected, *m_reconnecting;
    int m_retryCount = 0;
};
```

Resolved proof of structural safety: firing `connectionEstablished()` while the machine is in `Reconnecting` (not `Connecting`) produces no output, no crash, no incorrect change — that transition simply isn't in the graph from that state. This requires zero defensive code, unlike a switch statement where every handler must remember to guard against being called in the wrong state.

Resolved: guarded transitions via custom `QSignalTransition`, overriding `eventTest()` to inspect the actual signal arguments and decide, per-occurrence, whether a transition applies (e.g., only route “timeout” errors through a specific transition).

Key takeaways - `QStateMachine` makes valid transitions explicit declared edges — invalid transitions are no-ops by construction. - `entered()/exited()` on `QState` centralize state-specific setup/teardown. - Combine `QState::entered` with `QTimer::singleShot` for automatic, timed transitions (bounded auto-retry). - Custom `QSignalTransition::eventTest()` overrides route the same signal differently based on its argument data.

Day 26: Event Filters & Custom QEvent Subclasses

Theory: every notification mechanism in this course is built on `QEvent/event()` dispatch. `installEventFilter()` lets you observe (and optionally suppress) events destined for an object you don't own. Custom `QEvent` subclasses let you define arbitrary async application events, posted via the thread-safe `QCoreApplication::postEvent()`.

Resolved example — event filter on an object you don't control:

```
class ExternalObserver : public QObject
{
    Q_OBJECT
public:
    bool eventFilter(QObject *watched, QEvent *event) override
    {
        if (event->type() == QEvent::Timer)
            qDebug() << "[FILTER] observed a QEvent::Timer on" << watched;
        return false; // false: let the event continue to its normal destination
    }
};
// usage: poller->installEventFilter(observer);
```

`eventFilter()` runs before the target's own `event()` — Qt's dispatch checks installed filters first.

Resolved example — a custom, thread-safe posted event:


```

class SensorAlertEvent : public QEvent
{
public:
    static const QEvent::Type TypeId;
    SensorAlertEvent(const QString &id, double temp) : QEvent(TypeId), m_deviceId(id),
m_temperature(temp) {}
    QString deviceId() const { return m_deviceId; }
    double temperature() const { return m_temperature; }
private:
    QString m_deviceId; double m_temperature;
};
const QEvent::Type SensorAlertEvent::TypeId =
static_cast<QEvent::Type>(QEvent::registerEventType());

class AlertHandler : public QObject
{
protected:
    bool event(QEvent *e) override
    {
        if (e->type() == SensorAlertEvent::TypeId) { /* handle */ return true; }
        return QObject::event(e); // ALWAYS delegate unrecognized types to the base
    }
};

// QApplication::postEvent(handler, new SensorAlertEvent("sensor-07", 31.2));
// -- takes ownership, deletes automatically after dispatch; thread-safe to call from any
thread

```

Key takeaways - Signals are a convenience layer most Qt classes build atop the same `QEvent/event()` dispatch. - `installEventFilter()` addresses cross-cutting concerns (logging, legacy-code observation) signals/slots can't reach. - Custom `QEvent` subclasses + `postEvent()` give thread-safe, event-loop-queued delivery without a pre-declared signal/slot connection. - Always delegate unrecognized event types to `QObject::event(e)` — silently swallowing them breaks built-in behavior.

Day 27: QLoggingCategory & Idiomatic Qt Error Handling

Theory: Qt Core APIs consistently signal failure via checked return values, out-parameters, or error signals — never exceptions — because Qt's internals aren't universally exception-safe across the event loop and C-heritage code. Matching this convention in your own Qt-adjacent classes avoids mixing incompatible error-handling styles.

Resolved: named, filterable logging categories:

```
// logging.h
Q_DECLARE_LOGGING_CATEGORY(logSerial)
Q_DECLARE_LOGGING_CATEGORY(logNetwork)
// logging.cpp
Q_LOGGING_CATEGORY(logSerial, "mqtt_monitor.serial")
Q_LOGGING_CATEGORY(logNetwork, "mqtt_monitor.network")

// usage:
qCDebug(logSerial) << "Opening serial port /dev/ttyUSB0";           // silent by default
qCWarning(logSerial) << "Serial read timeout, retrying";           // shown by default
```

Enable verbose logging at runtime, no rebuild:

```
QT_LOGGING_RULES="mqtt_monitor.serial.debug=true" ./mqtt_monitor
```

Resolved: matching QFile's own error-handling convention in a custom class:

```
class DeviceReader : public QObject
{
public:
    bool openDevice(const QString &path)    // bool return + errorString(), like QFile::open()
    {
        if (!path.startsWith("/dev/")) {
            m_lastError = QString("Invalid device path format: %1").arg(path);
            qCWarning(logSerial) << m_lastError;
            return false;
        }
        return true;
    }
    QString lastError() const { return m_lastError; }
private:
    QString m_lastError;
};
```

Key takeaways - Qt Core signals failure via return values/out-params/error signals, never exceptions internally — match this convention in adjacent code. - `qCDebug` / `qCWarning` / `qCCritical` with a category argument give independently

filterable logging channels. - Debug-level category output is silent by default; enable per-category at runtime via `QT_LOGGING_RULES`. - Each subsystem (serial, network, database) should get its own category for fine-grained production log control.

Day 28: QTest — Unit Testing Qt Core Code, Including Async/Signal-Driven Code

Theory: most components in this course communicate via signals, not return values — a naive test asserting on a return value misses async behavior. `QSignalSpy` attaches to a signal and records emissions, with `wait()` for genuinely delayed signals.

Resolved example — synchronous logic, including data-driven tests:

```
class TestSerialLineParser : public QObject
{
    Q_OBJECT
private slots:
    void parsesValidLine()
    {
        SerialLineParser parser;
        auto result = parser.parse("SENSOR:07:TEMP:23.5");
        QVERIFY(result.has_value());
        QCOMPARE(result->deviceId, QString("sensor-07"));
        QCOMPARE(result->temperature, 23.5);
    }
    void rejectsGarbageLine()
    {
        SerialLineParser parser;
        QVERIFY(!parser.parse("GARBAGE DATA").has_value());
    }
    void parsesVariousFormats_data()
    {
        QTest::addColumn<QString>("input");
        QTest::addColumn<bool>("shouldSucceed");
        QTest::newRow("valid integer temp") << "SENSOR:01:TEMP:20" << true;
        QTest::newRow("non-numeric id") << "SENSOR:ab:TEMP:20" << false;
    }
    void parsesVariousFormats()
    {
        QFETCH(QString, input); QFETCH(bool, shouldSucceed);
        SerialLineParser parser;
        QCOMPARE(parser.parse(input).has_value(), shouldSucceed);
    }
};
QTEST_MAIN(TestSerialLineParser)
#include "test_seriallineparser.moc"
```

Resolved example — QSignalSpy for signal-driven and async code:

```

void notifySignalFiresOnChange()
{
    DeviceReading reading;
    QSignalSpy spy(&reading, &DeviceReading::temperatureChanged);
    reading.setTemperature(23.5);
    QCOMPARE(spy.count(), 1);
    QCOMPARE(spy.takeFirst().at(0).toDouble(), 23.5);
}

void notifySignalDoesNotFireOnNoOpChange() // guards against a Day-12-style regression
{
    DeviceReading reading;
    reading.setTemperature(23.5);
    QSignalSpy spy(&reading, &DeviceReading::temperatureChanged);
    reading.setTemperature(23.5); // same value again
    QCOMPARE(spy.count(), 0); // the no-op guard must suppress this
}

void waitForAsyncSignalWithTimeout()
{
    DeviceReading reading;
    QSignalSpy spy(&reading, &DeviceReading::temperatureChanged);
    QTimer::singleShot(100, [&reading]() { reading.setTemperature(99.9); });
    QVERIFY(spy.wait(500)); // blocks the TEST, pumping the event loop, until the signal
fires
}

```

If a later refactor accidentally removes the no-op guard, `notifySignalDoesNotFireOnNoOpChange()` fails immediately (`spy.count()` becomes 1) — catching the exact regression Day 12 warned was easy to introduce.

Key takeaways - `QTEST_MAIN` + `#include "file.moc"` generates a runnable test binary; each private slot becomes an auto-discovered test case. - `QCOMPARE` gives actual-vs-expected output on failure; prefer it over bare `QVERIFY(a==b)`. - Data-driven tests (`_data()` + `QFETCH`) avoid duplicated near-identical test methods. - `QSignalSpy` (with `wait()` for delayed emissions) is the correct tool for asserting on signal-driven, including async, behavior. - Write tests specifically for subtle correctness properties (no-op guards, anchored-regex rejection) that are easy to silently regress.

Day 29: CMake Project Structure & Deployment for Embedded Linux Targets

Theory: a real project separates shared library code (built once) from multiple executables (app + tests) linking against it, and must support cross-compilation without restructuring.

Resolved layout:

```
mqtt_monitor/  
├── CMakeLists.txt  
├── src/          (library: parser, model, config loader, serializer)  
├── app/          (the actual service executable)  
├── tests/        (QtTest binaries, linking the library)  
└── cmake/toolchain-rpi.cmake
```

```
# src/CMakeLists.txt  
add_library(mqtt_monitor_core STATIC seriallineparser.cpp devicereading.cpp configloader.cpp  
readingserializer.cpp)  
target_include_directories(mqtt_monitor_core PUBLIC ${CMAKE_CURRENT_SOURCE_DIR})  
target_link_libraries(mqtt_monitor_core PUBLIC Qt6::Core Qt6::Network)  
# PUBLIC: consumers automatically inherit include path AND the Qt6 dependency  
  
# app/CMakeLists.txt  
add_executable(mqtt_monitor main.cpp)  
target_link_libraries(mqtt_monitor PRIVATE mqtt_monitor_core)  
  
# tests/CMakeLists.txt  
add_executable(test_seriallineparser test_seriallineparser.cpp)  
target_link_libraries(test_seriallineparser PRIVATE mqtt_monitor_core Qt6::Test)  
add_test(NAME SerialLineParserTests COMMAND test_seriallineparser)
```

Source files compile exactly once into the library, regardless of how many executables use them.

Resolved cross-compilation toolchain (`cmake/toolchain-rpi.cmake`):

```
set(CMAKE_SYSTEM_NAME Linux)  
set(CMAKE_SYSTEM_PROCESSOR arm)  
set(CMAKE_C_COMPILER arm-linux-gnueabi-gcc)  
set(CMAKE_CXX_COMPILER arm-linux-gnueabi-g++)  
set(CMAKE_SYSROOT /opt/rpi-sysroot)  
set(CMAKE_FIND_ROOT_PATH ${CMAKE_SYSROOT})  
set(CMAKE_FIND_ROOT_PATH_MODE_PROGRAM NEVER) # moc runs on the HOST, not the target  
set(CMAKE_FIND_ROOT_PATH_MODE_LIBRARY ONLY) # libraries/headers ONLY from the ARM sysroot  
set(CMAKE_FIND_ROOT_PATH_MODE_INCLUDE ONLY)  
set(CMAKE_FIND_ROOT_PATH_MODE_PACKAGE ONLY)
```

```
cmake -B build-rpi -S . -DCMAKE_TOOLCHAIN_FILE=cmake/toolchain-rpi.cmake
cmake --build build-rpi
```

`moc` (code generation) must run on the host during the build; the Qt6 libraries/headers linked against must be the ARM versions matching the Pi's runtime. Getting `PROGRAM / LIBRARY` modes backwards either breaks `moc` entirely or links the wrong-architecture Qt libraries.

Resolved systemd deployment, tying back to Day 2:

```
[Service]
ExecStart=/usr/local/bin/mqtt_monitor
Restart=on-failure
RestartSec=5
# systemd sends SIGTERM on stop, escalates to SIGKILL after TimeoutStopSec --
# exactly why Day 2's SIGTERM handler calling quit() matters: without it,
# systemd eventually SIGKILLS the process, skipping cleanup (e.g. Day 7's log flush).
```

Key takeaways - Structure as a shared library linked by the app and every test binary — one source of truth for build flags. - `PUBLIC` propagates a dependency to consumers; `PRIVATE` keeps it internal. - Cross-compiling requires a toolchain file with a target compiler and sysroot, while still finding build-time tools like `moc` on the host. - Day 2's graceful-shutdown handling is what makes a systemd-managed service actually flush data on restart/stop rather than being SIGKILLED after a timeout.

Day 30 (Capstone): Qt Core Serial → JSON → Network Relay

Full synthesis: a worker thread (Days 15–16) simulating serial ingestion, parsing (Day 11/14) into validated model objects (Day 12/13), a state machine (Day 25) managing the network connection lifecycle, JSON serialization (Day 10) for outgoing messages, TCP transport (Day 22) with proper framing, and structured logging (Day 27) throughout.

Architecture:

SerialSimulatorWorker (own QThread)		NetworkRelay (main thread)
generates fake lines on a timer	---->	receives readings via queued signal
parses each line, emits		QStateMachine: Disconnected -> Connecting
readingParsed(DeviceReading*)		-> Connected -> Reconnecting
		buffers readings while disconnected
		sends framed JSON once connected

```
// serialsimulatorworker.h
class SerialSimulatorWorker : public QObject
{
    Q_OBJECT
public slots:
    void initialize() // connected to QThread::started, NOT the constructor
    {
        connect(&m_timer, &QTimer::timeout, this, &SerialSimulatorWorker::generateAndParse);
        m_timer.setTimerType(Qt::PreciseTimer);
        m_timer.start(400);
    }
    void shutdown() { m_timer.stop(); emit finished(); }
signals:
    void readingParsed(DeviceReading *reading); // ownership transfers to the receiver
    void finished();
private slots:
    void generateAndParse()
    {
        QString rawLine = /* simulated SENSOR:NN:TEMP:XX.X */;
        auto parsed = m_parser.parse(rawLine);
        if (!parsed) { qWarning(logSerial) << "Rejected malformed line"; return; }
        auto *reading = new DeviceReading(); // no parent -- crossing threads next
        reading->setDeviceId(parsed->deviceId);
        reading->setTemperature(parsed->temperature);
        reading->setTimestamp(QDateTime::currentDateTimeUtc()); // always UTC
        emit readingParsed(reading); // queued delivery across threads
    }
private:
    SerialLineParser m_parser;
    QTimer m_timer;
};
```



```

// networkrelay.h
class NetworkRelay : public QObject
{
    Q_OBJECT
public:
    NetworkRelay(const QString &host, quint16 port, QObject *parent = nullptr)
        : QObject(parent), m_host(host), m_port(port)
    {
        connect(&m_socket, &QTcpSocket::connected, this, &NetworkRelay::connectionSucceeded);
        connect(&m_socket, &QTcpSocket::errorOccurred, this, &NetworkRelay::connectionLost);
        connect(&m_socket, &QTcpSocket::disconnected, this, &NetworkRelay::connectionLost);
        setupStateMachine();
        m_machine.start();
    }
public slots:
    void start() { emit connectRequested(); }
    void enqueueReading(DeviceReading *reading) // connected via queued cross-thread signal
    {
        QJsonObject json = ReadingSerializer::toJson(*reading);
        m_pendingQueue.enqueue(json);
        reading->deleteLater(); // worker handed us ownership; we're done with it now
        if (m_connected) flushQueue();
    }
signals:
    void connectRequested(); void connectionSucceeded();
    void connectionLost(); void retryTimeElapsed();
private:
    void setupStateMachine()
    {
        m_disconnected = new QState(&m_machine);
        m_connecting = new QState(&m_machine);
        m_connected_ = new QState(&m_machine);
        m_reconnecting = new QState(&m_machine);

        connect(m_connecting, &QState::entered, [this] { m_socket.connectToHost(m_host,
m_port); });
        connect(m_connected_, &QState::entered, [this] { m_connected = true; m_retryCount =
0; flushQueue(); });
        connect(m_reconnecting, &QState::entered, [this] {
            ++m_retryCount; m_connected = false;
            int delayMs = qMin(1000 * m_retryCount, 10000); // capped exponential backoff
            QTimer::singleShot(delayMs, this, &NetworkRelay::retryTimeElapsed);
        });

        m_disconnected->addTransition(this, &NetworkRelay::connectRequested, m_connecting);
        m_connecting->addTransition(this, &NetworkRelay::connectionSucceeded, m_connected_);
        m_connecting->addTransition(this, &NetworkRelay::connectionLost, m_reconnecting);
        m_connected_->addTransition(this, &NetworkRelay::connectionLost, m_reconnecting);
        m_reconnecting->addTransition(this, &NetworkRelay::retryTimeElapsed, m_connecting);
        m_machine.setInitialState(m_disconnected);
    }
    void flushQueue() { while (!m_pendingQueue.isEmpty())
sendFramed(m_pendingQueue.dequeue()); }
    void sendFramed(const QJsonObject &jjson) // length-prefix framing, per Day 22
    {
        QByteArray payload = QJsonDocument(jjson).toJson(QJsonDocument::Compact);
        QByteArray frame;
        QDataStream stream(&frame, QIODevice::WriteOnly);
        stream.setVersion(QDataStream::Qt_6_5);
    }
}

```

```

        stream << quint32(payload.size());
        frame.append(payload);
        m_socket.write(frame);
    }
    QTcpSocket m_socket; QStateMachine m_machine;
    QState *m_disconnected, *m_connecting, *m_connected_, *m_reconnecting;
    QString m_host; quint16 m_port;
    bool m_connected = false; int m_retryCount = 0;
    QQueue<JsonObject> m_pendingQueue;
};

```

```

// main.cpp
int main(int argc, char *argv[])
{
    QApplication app(argc, argv);
    std::signal(SIGINT, handleTermination);    // graceful shutdown, per Day 2/29
    std::signal(SIGTERM, handleTermination);

    auto *relay = new NetworkRelay("localhost", 9999, &app);
    auto *workerThread = new QThread();
    auto *worker = new SerialSimulatorWorker();    // no parent -- moving across threads

    worker->moveToThread(workerThread);
    connect(workerThread, &QThread::started, worker, &SerialSimulatorWorker::initialize);
    connect(worker, &SerialSimulatorWorker::readingParsed, relay,
    &NetworkRelay::enqueueReading);
    connect(worker, &SerialSimulatorWorker::finished, workerThread, &QThread::quit);
    connect(workerThread, &QThread::finished, worker, &QObject::deleteLater);
    connect(workerThread, &QThread::finished, workerThread, &QObject::deleteLater);
    connect(&app, &QCoreApplication::aboutToQuit, [worker]() {
        QMetaObject::invokeMethod(worker, "shutdown", Qt::QueuedConnection);
    });

    workerThread->start();
    relay->start();
    return app.exec();
}

```

Resolved sample behavior: while the relay target isn't yet reachable, the state machine cycles Disconnected → Connecting → Reconnecting with capped exponential backoff, buffering incoming readings in `m_pendingQueue`; the instant the target becomes reachable, the machine enters Connected and flushes the entire buffer as framed JSON messages, then continues sending new readings as they arrive.

Architectural walkthrough — every day's contribution: - **Days 15-16:** the worker lives on its own thread, correctly initialized/shut down via the started/finished chain. - **Day 11/14:** raw lines parsed via the same anchored-regex parser, rejecting malformed input. - **Day 12/13:** `DeviceReading` holds validated, always-UTC-timestamped state. - **Day 15:** `readingParsed` crosses threads via an auto-detected queued connection — ownership transfers cleanly, no shared-memory race. - **Day 25:** the connection lifecycle is a declared transition graph — an out-of-

order event is structurally a no-op. - **Day 10/22:** outgoing readings are JSON-serialized and length-prefix framed for reliable TCP reassembly. - **Buffering:** readings during Reconnecting are queued, not dropped — directly analogous to Day 7's buffer-and-flush discipline. - **Day 2/29:** SIGTERM/SIGINT trigger graceful shutdown through a queued cross-thread invocation, exactly what a systemd deployment depends on. - **Day 27:** every component logs through its own filterable category.

From here, the natural next steps are replacing the simulated worker with a real `QSerialPort` reader, and the demo TCP target with an actual MQTT client library — the architecture around it doesn't need to change.